The Web Conference · WWW '24 · Singapore

SimpleDyG

On the Feasibility of Simple Transformer for Dynamic Graph Modeling

Yuxia Wu Yuan Fang Lizi Liao

Nhóm 6: Vũ Duy Thụ & Đỗ Quốc Thịnh

1. INTRODUCTION · BỐI CẢNH

Đồ thị động trong thực tế

Các đỉnh và cạnh xuất hiện và biến đổi liên tục theo thời gian.

Mạng xã hội · UCI

Nút là người dùng, cạnh là tin nhắn; cạnh mới liên tục xuất hiện.

Hệ gợi ý · ML-10M

Người dùng gắn thẻ cho phim theo thời gian.

Mạng trích dẫn · Hội thoại

Bài báo mới trích dẫn bài cũ; các lượt thoại nối tiếp nhau.

ĐỊNH NGHĨA HÌNH THỨC

$$G = (V, E, T, X)$$

cạnh (v_i, v_j, τ), $\tau \in T$

V: tập nút
E: tập cạnh (tương tác)
T: miền thời gian
X: đặc trưng đầu vào
v_i, v_j: hai nút tương tác
τ: thời điểm tương tác

1. INTRODUCTION · VẤN ĐỀ ĐẶT RA

Dynamic Link Prediction

1

Input

Chuỗi cạnh (v_i, v_j, τ), chia thành T time step; lịch sử đến bước T−1 làm đầu vào

↓

2

Model

SimpleDyG (tham số θ)

↓

3

Output

Tập nút liên kết với ego-node tại bước T (dynamic link prediction)

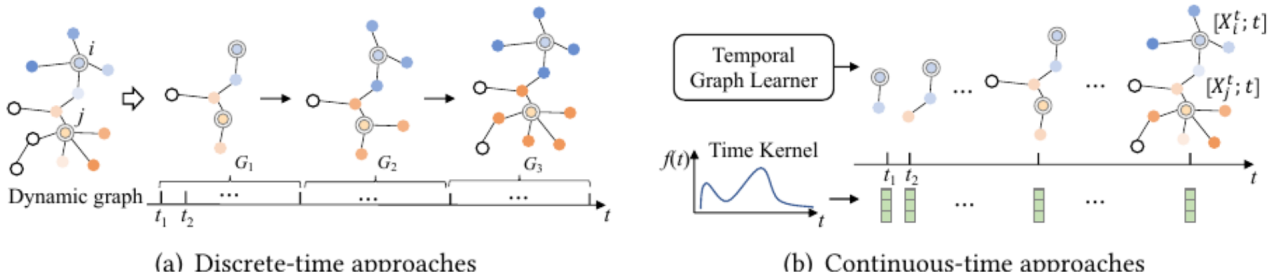
PHÁT BIỂU BÀI TOÁN

Cho trước lịch sử tương tác của nút v_i trước bước thời gian T, dự đoán tập các nút sẽ kết nối với v_i tại bước T:

$$\{ v_i^{T,1} , v_i^{T,2} , \dots \}$$

Chia dữ liệu: T = test · T−1 = validation · còn lại = train **Đánh giá:** NDCG@5, Jaccard

Hai hướng tiếp cận hiện có



(a) Discrete-time approaches (b) Continuous-time approaches

	Discrete-time	Continuous-time
Cách nhìn	Chuỗi snapshot đồ thị	Mỗi tương tác là một sự kiện có timestamp
Module	GNN + RNN/attention (DySAT, EvolveGCN)	Random walk, time encoding, point process (TGAT, TGN, TREND, GraphMixer)
Điểm yếu	Mất hoàn toàn thông tin thời gian chi tiết trong snapshot	Khó nắm bắt phụ thuộc dài hạn

Hạn chế của các phương pháp hiện có

1. Thời gian: thô hoặc ngắn hạn

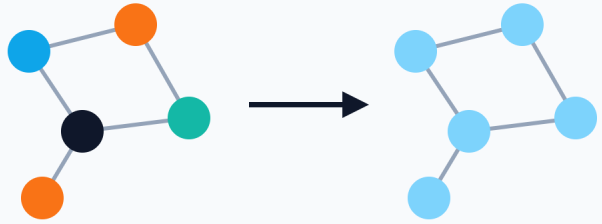
Discrete-time

Bỏ mất thời điểm chi tiết trong snapshot.

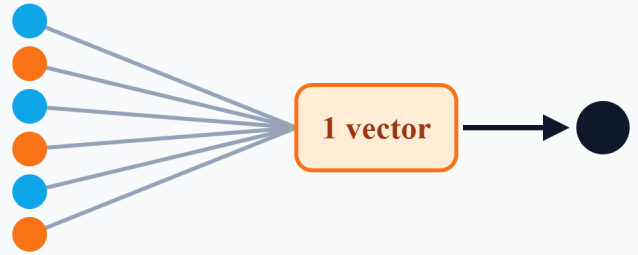
Continuous-time

Giữ chi tiết nhưng khó nhớ lịch sử dài.

2. Hạn chế của message-passing GNN (càng sâu càng nặng)



Over-smoothing: biểu diễn các nút dần trở nên giống nhau.



Over-squashing: thông tin từ vùng lân cận rộng bị nén vào một vector cố định.

Transformer thuần có đủ cho đồ thị động không?

Vì sao chọn Transformer?

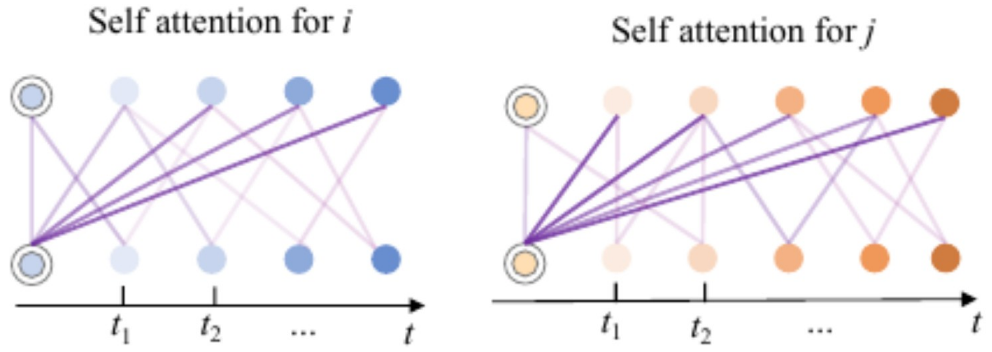
1. Chuỗi liên tục

Nhận trực tiếp chuỗi dữ liệu, không cần chia snapshot rời rạc.

2. Self-attention

Bắt phụ thuộc dài hạn, ít bị over-smoothing và over-squashing hơn.

Transformer gốc, không sửa kiến trúc.



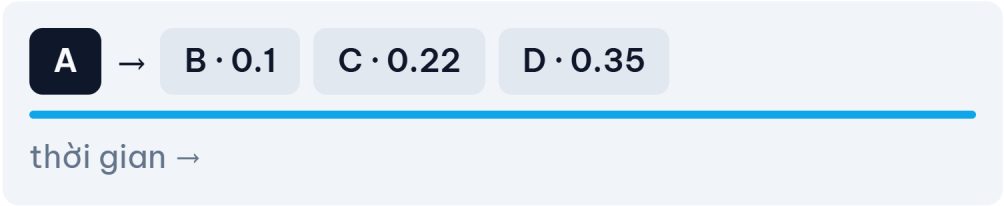
(c) Self-attention in Transformer

Hình 1(c), nguồn: bài báo gốc

Thách thức và giải pháp của SimpleDyG

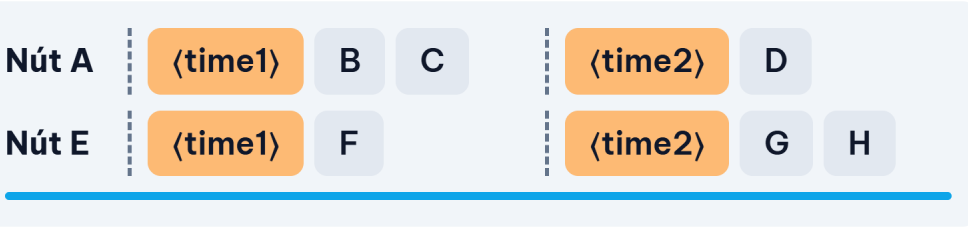
Thách thức 1 · Scalability → **Temporal ego-graph**

Attention theo cặp chỉ xử lý được đồ thị nhỏ; mỗi nút chỉ giữ chuỗi lịch sử của riêng mình.



Thách thức 2 · Căn chỉnh thời gian → **Temporal token**

10 tương tác trong 1 giờ khác 10 tương tác trong 1 ngày; token dùng chung đánh dấu time step.



2. RELATED WORK

Dynamic Graph Learning

Discrete-time

Cách làm: chuỗi snapshot, GNN kết hợp RNN hoặc attention

DySAT: graph attention + self-attention

EvolveGCN: RNN cập nhật tham số GCN

Điểm yếu: thiếu thông tin thời gian chi tiết

Continuous-time

Cách làm: xét từng sự kiện theo timestamp

Walk: temporal random walk, causal anonymous walk

Time encoding: TGAT, TGN, GraphMixer

Point process: DyRep, TREND

Điểm yếu: khó nắm phụ thuộc dài hạn

KHÁC BIỆT CỦA SIMPLEDYG

Xử lý phụ thuộc dài hạn nhờ Transformer, và giữ đầy đủ thứ tự thời gian của từng chuỗi.

Transformer cho đồ thị

STATIC

1. Trên đồ thị tĩnh (static)

Graphormer, SAN, Pure Transformers tích hợp tri thức cấu trúc (spatial, distance, centrality encoding) vào Transformer.

DYNAMIC

2. Trên đồ thị động (phức tạp)

Yu et al. (NeurIPS '23) cần thiết kế mô-đun **rất phức tạp** để mã hóa hàng xóm và khoảng thời gian.

SỰ KHÁC BIỆT VƯỢT TRỘI CỦA SIMPLEDYG

Đơn giản nhưng vượt trội

✓ **Đơn giản:** không thêm mô-đun phức tạp.

✓ **Vanilla Transformer:** giữ nguyên kiến trúc gốc.

✓ **Đổi mới ở đầu vào:** biến đổi đồ thị thành chuỗi token.

Chuỗi Input và Output

Token cấu trúc | Token thời gian | Token nút (nội dung)

Input x_i: lịch sử của ego-node v_i

⟨hist⟩ v_i ⟨time1⟩ a b ⟨time2⟩ c ⟨endofhist⟩

Các bước thời gian 1 đến T−1, mỗi bước mở đầu bằng token thời gian.

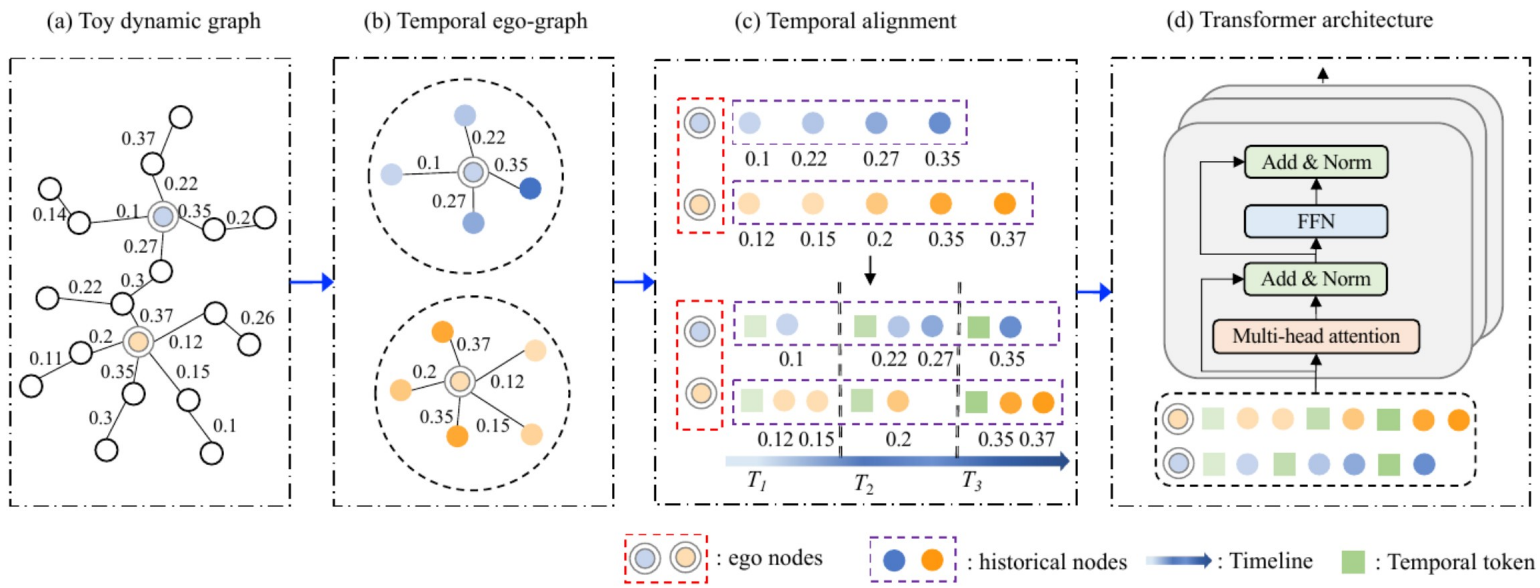
Output y_i: các nút liên kết ở bước T

⟨pred⟩ ⟨timeT⟩ d e ⟨endofpred⟩

Mô hình dừng dự đoán khi sinh ra ⟨endofpred⟩, nên tự quyết số nút.

Huấn luyện: ghép [x; y], dự đoán token kế tiếp (negative log-likelihood).

Tổng quan kiến trúc SimpleDyG



1. Đồ thị động → 2. Temporal Ego-graph → 3. Temporal Alignment → **4. Transformer**

KẾT LUẬN

Đóng góp chính và kết quả

1 **Khả thi**
Transformer gốc, đơn giản mà hiệu quả cho đồ thị động.

2 **Biến đổi đầu vào**
Temporal ego-graph và temporal token.

3 **Thực nghiệm**
4 tập dữ liệu thực: UCI, ML-10M, Hepth, MMConv.

Tập · Metric	GraphMixer	DySAT	SimpleDyG
UCI · Jaccard	0.042	0.010	**0.092**
UCI · NDCG@5	0.104	0.010	**0.104 (ngang bằng)**
ML-10M · NDCG@5	0.081	0.058	**0.138**
ML-10M · Jaccard	0.043	0.050	**0.131**

Thiết lập tái lập chi tiết

Dataset	# Nút · # Cạnh	Time step	Truy vấn train / val / test	SimpleDyG (theo script tác giả)	Tham số
UCI · Social	1.781 · 16.743	13 (T−1 val, T test)	1.708 / 146 / 110	6 lớp · 8 head · dim 768 · batch 32 · lr 1e-4	44,7 triệu
ML-10M · Rating	15.841 · 48.561	13 (T−1 val, T test)	13.701 / 738 / 396	2 lớp · 4 head · dim 128 · batch 64 · lr 1e-5	2,6 triệu

Huấn luyện SimpleDyG

- AdamW của tác giả, lr giảm tuyến tính, tối đa 100 epoch.
- Early stopping theo NDCG@5 validation, *patience* 10 (corrected: 20).
- Chuỗi train ≤ 512 token, context 1.024; micro-batch 8 + gradient accumulation, giữ nguyên batch logic.
- Dự đoán: greedy decoding đến token kết thúc.

Baseline (DyGLib)

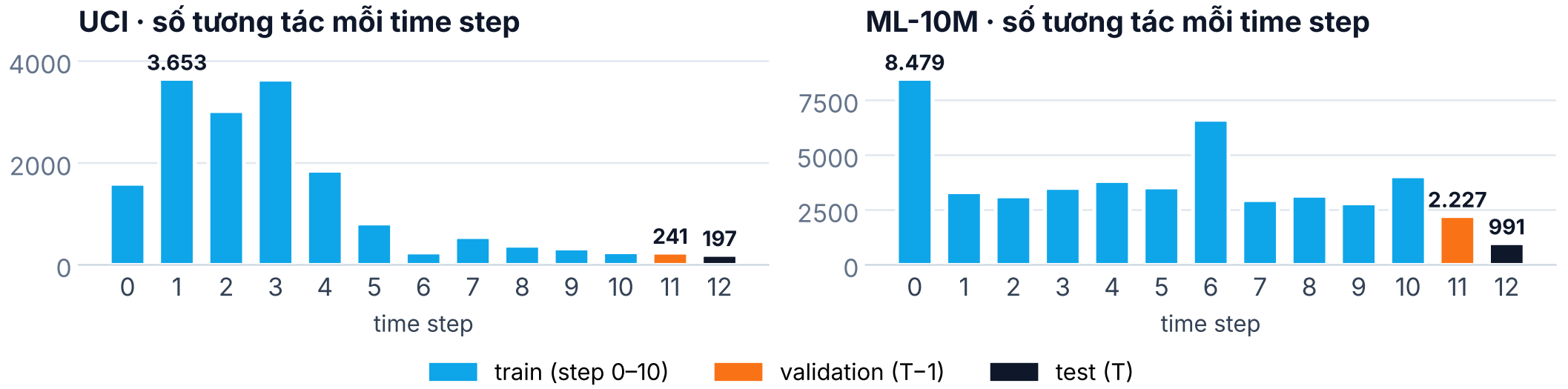
- TGAT (20 neighbour) và GraphMixer (1.024 token lịch sử).
- Loss BPR, feature nút/cạnh = 0, batch 128.
- Chọn dim ∈ {16, 32, 64, 128, 256, 512} theo validation → 6 lần huấn luyện mỗi seed.

Đánh giá

- NDCG@5 chuẩn (khử trùng lặp) và Jaccard.
- 10 seed (0–9), báo cáo mean ± std; so biến thể theo cặp seed, CI 95% (t) và sign test.
- Kiểm tra hash dữ liệu và split trước khi chạy.

Biến thể temporal token: `full` gốc `same time` một token cho mọi slot `no time` bỏ token `per-ego permute` xáo nhãn riêng từng ego

Dữ liệu thay đổi mạnh theo thời gian



1 UCI thưa dần

Từ ~3.600 tương tác/step còn **197** ở step test (**110 user**) → tập test nhỏ, điểm dao động giữa seed.

2 ML-10M ổn định hơn

~3.000–4.000 tương tác/step, có đỉnh ở step 0 và 6; step test còn 991 tương tác của 396 user.

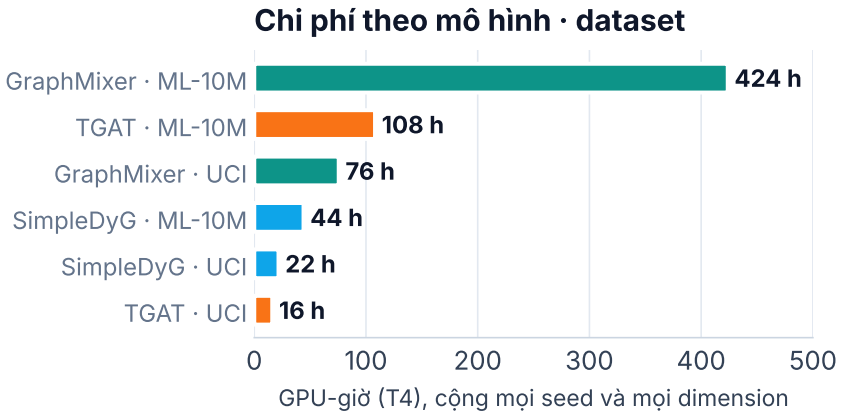
3 Lập luận của paper

Paper (Hình A.1) dùng các thay đổi đột ngột này để biện minh cho temporal alignment; slide 23 kiểm tra lại.

Chi phí chạy trên Kaggle 2×T4

Đợt	Nội dung	Lượt	GPU-giờ	2×T4
1	2 dataset × 5 cấu hình × 10 seed	100	670	**≈ 335 giờ**
2	UCI per-ego permute · 10 seed	10	6,4	**≈ 3,2 giờ**
3	ML-10M corrected · 3 × 5 seed	15	12,6	**≈ 6,5 giờ**
Tổng		**125**	**≈ 689**	**≈ 345 giờ**

GPU-giờ = tổng thời gian huấn luyện và test ghi trong log, gồm cả decode validation mỗi epoch. Thời gian 2×T4 = GPU-giờ / 2, chưa gồm cài đặt và nạp lại checkpoint giữa các phiên.



Hạ tầng

Kaggle 2 × Tesla T4 (16 GB), PyTorch 2.11 / CUDA 12.8. Hai worker độc lập, mỗi GPU một lượt; checkpoint 5 phút/lần, phiên ≤ 11,75 giờ rồi chạy tiếp.

< 10%
tổng GPU-giờ dành cho SimpleDyG (≈ 66 / 689).

47 s
/epoch SimpleDyG trên UCI, so với 67 s (TGAT) và 259 s (GraphMixer).

4. THỰC NGHIỆM · TÁI LẬP TABLE 2

Kết quả so với paper

Tập · Metric	TGAT	paper	GraphMixer	paper	SimpleDyG (chúng tôi)	paper
UCI · NDCG@5	.049 ± .024	.061	.070 ± .017	.104	**.104 ± .017**	.104
UCI · Jaccard	.028 ± .013	.020	.034 ± .009	.042	**.087 ± .013**	.092
ML-10M · NDCG@5	.009 ± .014	.066	.152 ± .046	.081	**.099 ± .061**	.138
ML-10M · Jaccard	.005 ± .006	.021	.086 ± .046	.043	**.093 ± .057**	.131

Mean ± std trên 10 seed, giao thức author_compat. Jaccard của baseline = max theo k ∈ {1, 5, 10, 20} trên test như paper; SimpleDyG dùng tập sinh ra. Chỉ chạy 2/8 baseline của paper.

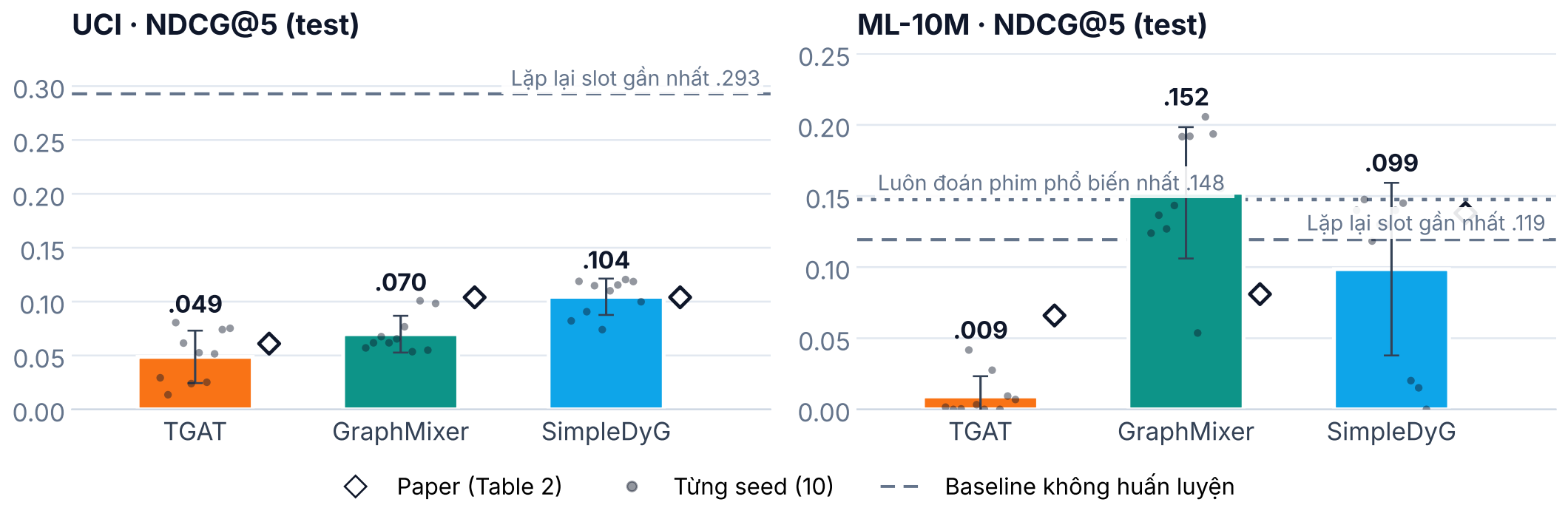
✓ UCI · TÁI LẬP ĐƯỢC

SimpleDyG đạt **.104**, đúng bằng paper, và tốt nhất trong ba mô hình ở cả hai metric. Std giữa seed (.017) tương đương paper.

× ML-10M · LỆCH SO VỚI PAPER

SimpleDyG thấp hơn paper **.039**, std lớn gấp ~6 lần; GraphMixer lại "vượt" paper. Nguyên nhân: mô hình **suy biến** về dự đoán phổ biến (slide 19).

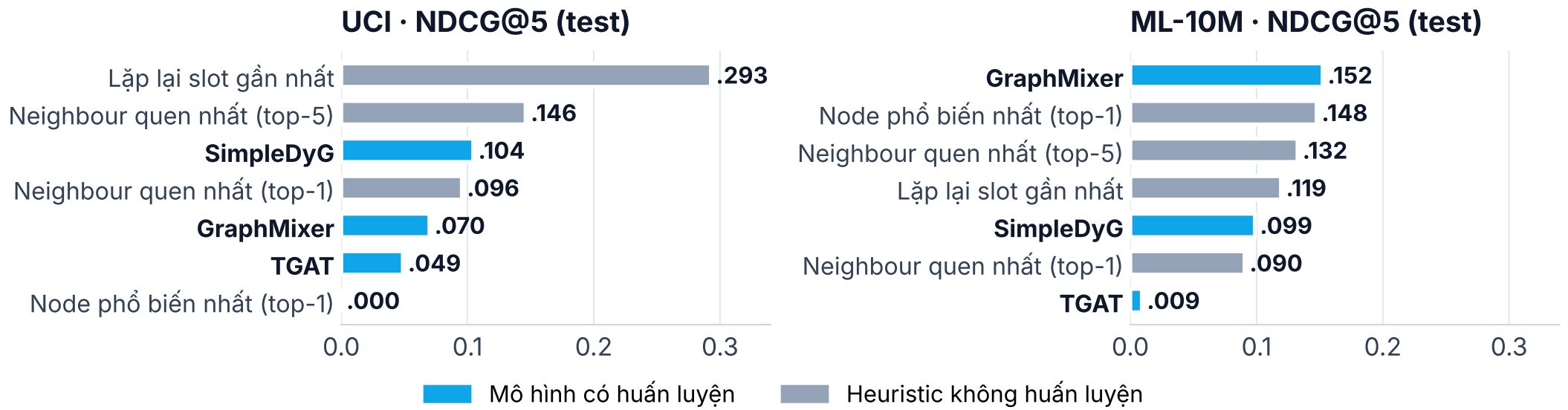
Mô hình, paper và baseline không huấn luyện



UCI: heuristic "lặp lại slot gần nhất" (**.293**) cao gấp ~2,8 lần mọi mô hình. **ML-10M:** không mô hình nào vượt "luôn đoán phim phổ biến nhất" (**.148**).

PHÁT HIỆN 1 · BASELINE ĐƠN GIẢN

Heuristic không huấn luyện mạnh hơn mô hình



2,8× "Lặp lại slot gần nhất" so với SimpleDyG trên UCI (.293 vs .104); vẫn giữ trên validation (.219).

Vì sao?

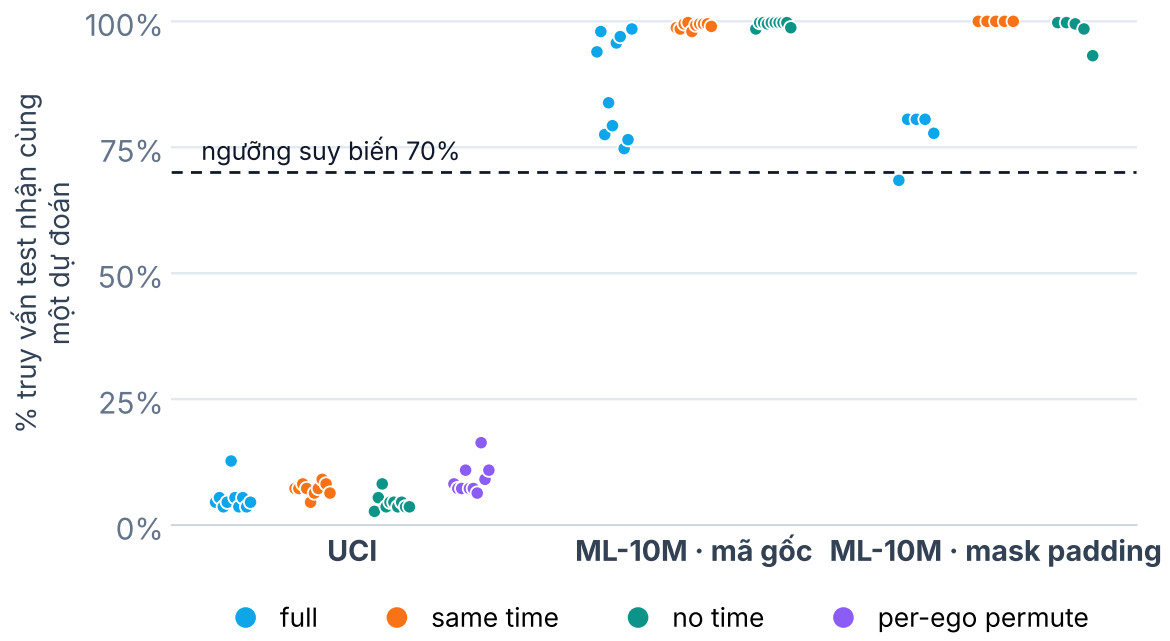
UCI có tính lặp lại rất mạnh: người dùng nhắn lại đúng những người vừa nhắn ở slot trước.

Hàm ý

Paper không có baseline này; cần luôn báo cáo kèm khi đánh giá dynamic link prediction.

PHÁT HIỆN 2 · ML-10M SUY BIẾN

ML-10M: mô hình chỉ đoán một bộ phim



1 Phim 1419

Đích phổ biến nhất khi train (1.417 lần), có trong ground truth của 17,7% user test. 23/30 lượt mã gốc đoán phim này cho 75–100% user.

2 Trần điểm = độ phổ biến

Luôn đoán 1419 cho NDCG@5 **.148** / Jaccard **.140**, đúng mức cao nhất của mọi lượt SimpleDyG và GraphMixer.

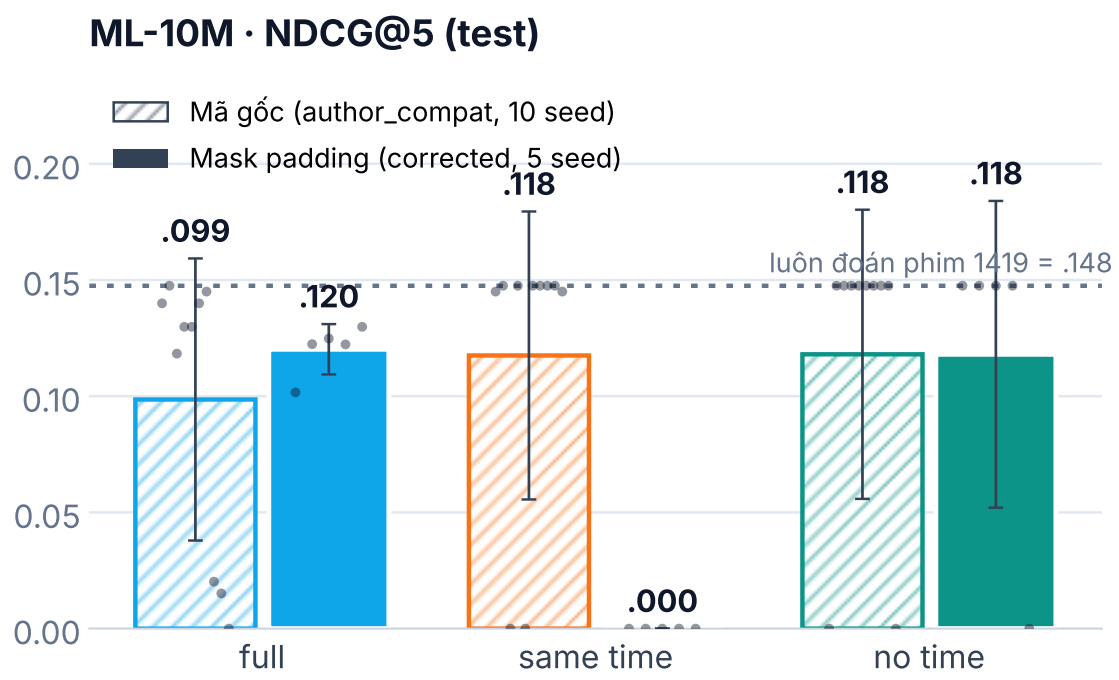
3 Trùng với paper

Paper báo ML-10M same time = **.147 ± .001**, gần như đúng mức này (suy luận từ số liệu).

UCI: **63–96** tập dự đoán khác nhau trên 110 truy vấn → dự đoán cá nhân hoá. ML-10M: chỉ **2–9** tập trên 396 truy vấn.

CHẨN ĐOÁN · ML-10M

Mask padding: học sớm hơn nhưng vẫn suy biến



Giả thuyết

Loss tính cả trên padding, lr 1e-5: NDCG val = 0 suốt ~20 epoch → *patience* 10 dừng sớm.

Thử nghiệm: protocol corrected

Mask padding + attention mask, *patience* 20, 3 biến thể × 5 seed.

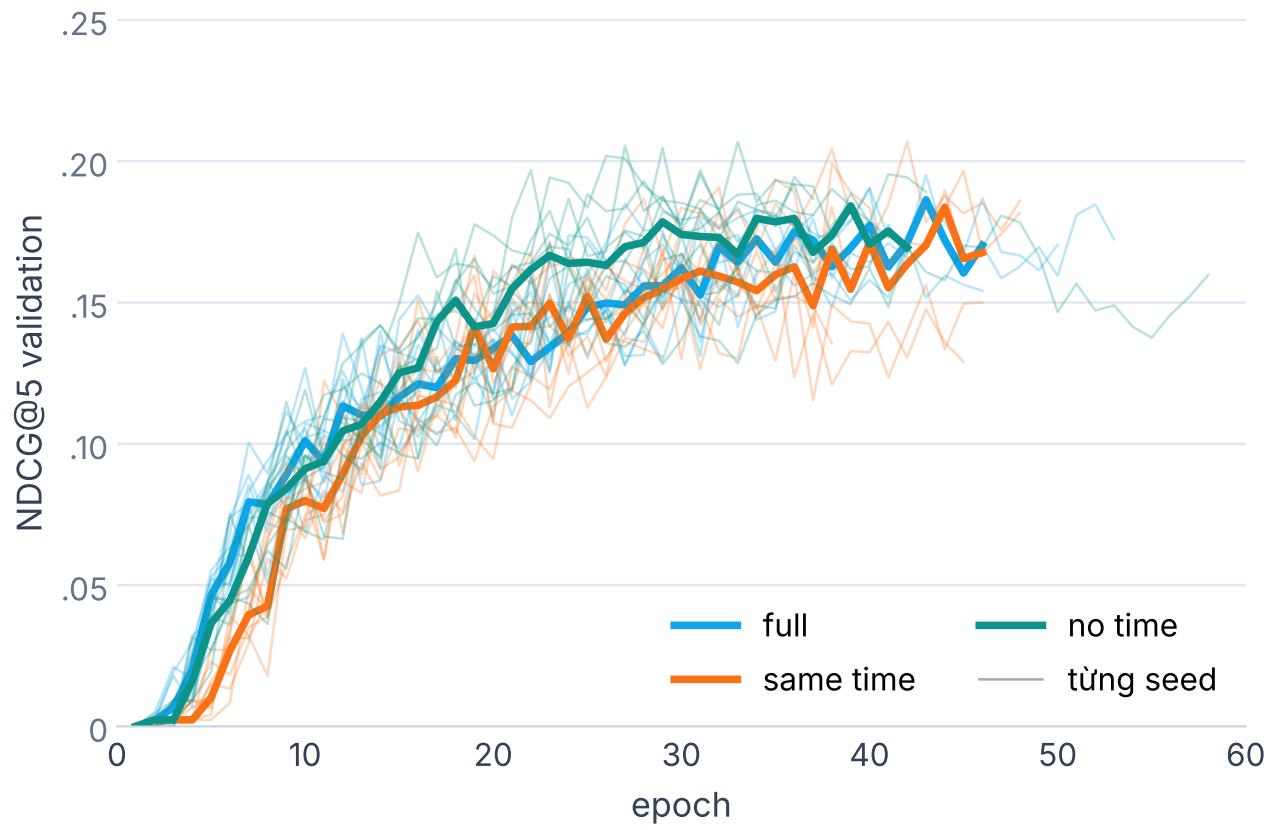
Kết quả

- full ổn định hơn: std **.061 → .011**, không còn seed = 0.
- Nhưng .120 < .148, **14/15** lượt vẫn suy biến; same time: 5/5 = 0.

Kết luận: với cấu hình paper (2 lớp, dim 128, lr 1e-5), ML-10M **bị chi phối bởi độ phổ biến** → không dùng để kết luận về temporal alignment.

CHẨN ĐOÁN · QUÁ TRÌNH HỌC (1/2)

UCI: mô hình học bình thường



1 Học ngay từ đầu

Điểm rời 0 từ epoch ~4 và tăng đều, không có giai đoạn "đứng yên".

2 Bão hoà ở .15–.18

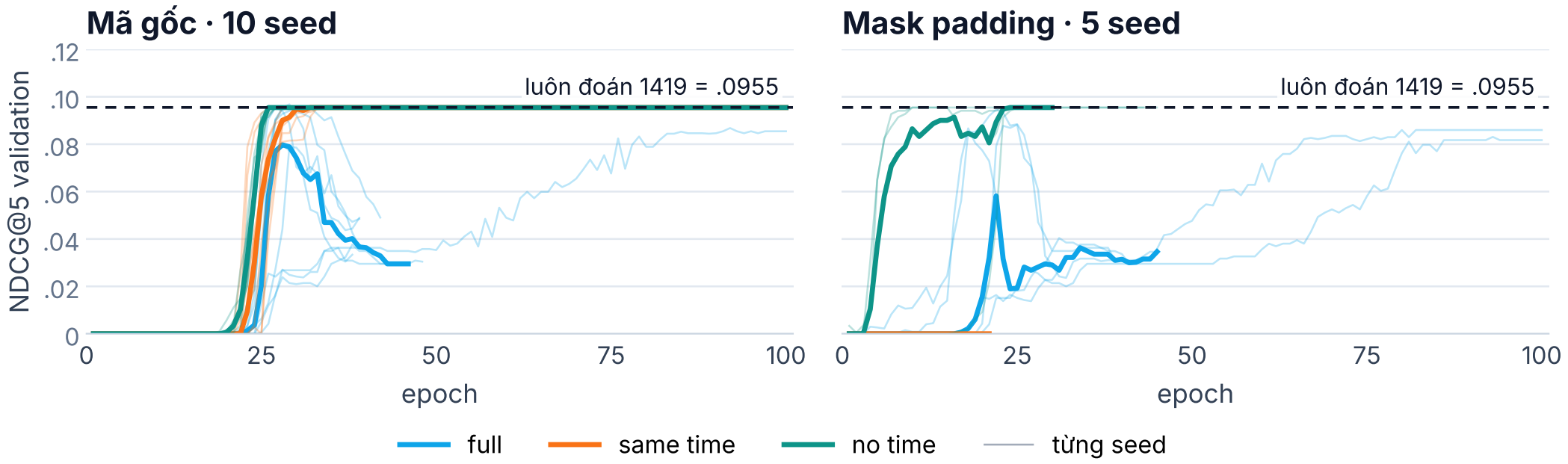
Sau ~30 epoch điểm chững lại; early stopping dừng ở epoch 20–58.

3 Ba biến thể chồng nhau

Ba biến thể học gần như giống hệt nhau → temporal token không đổi cách học.

CHẨN ĐOÁN · QUÁ TRÌNH HỌC (2/2)

ML-10M: kẹt ở độ phổ biến, kể cả khi mask padding



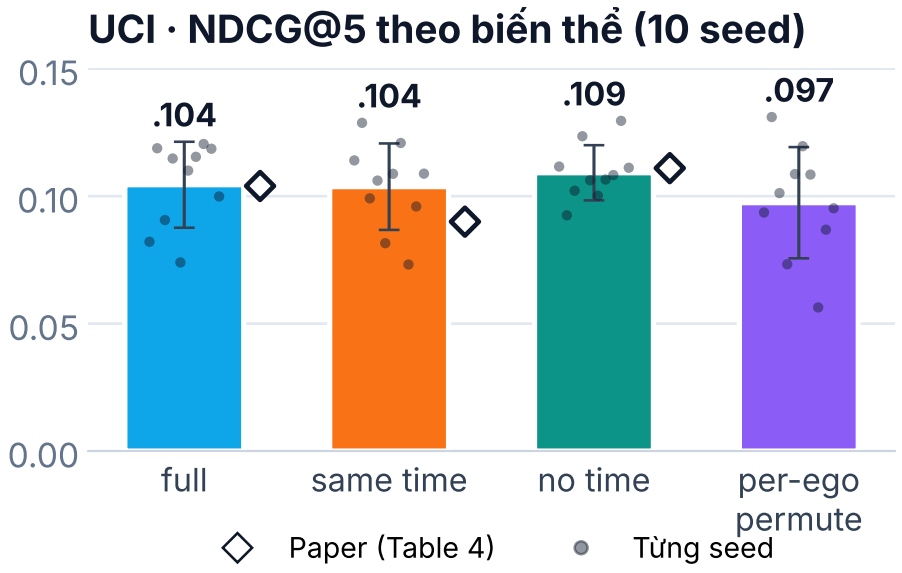
1 Mã gốc: 0 rồi nhảy vọt

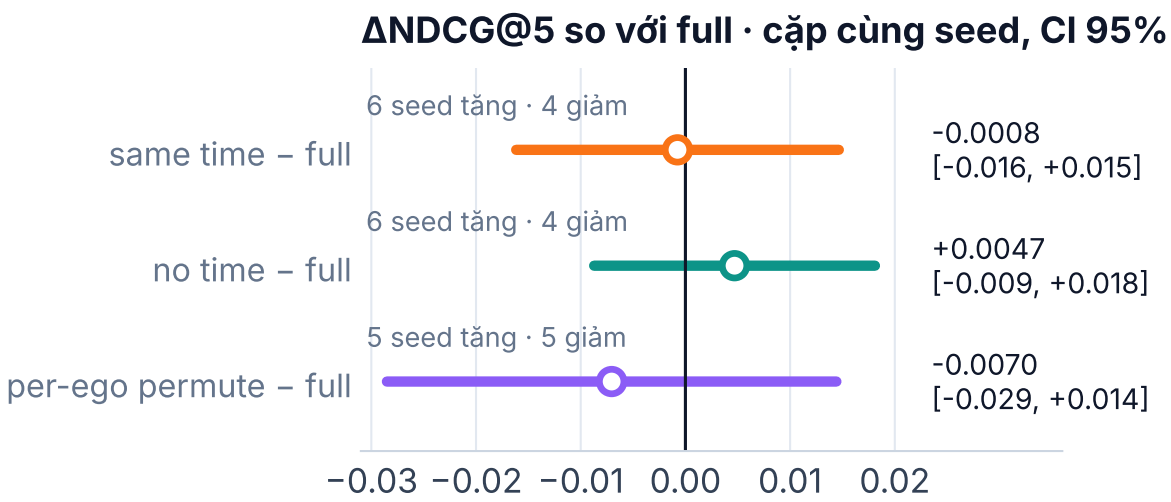
Điểm = 0 suốt ~22–25 epoch, rồi nhảy lên **đúng .0955** và đứng yên (20/30 lượt). full rời đỉnh thì tụt về ~.03.

2 Mask padding: sớm hơn, cùng đích

no time học từ epoch **4** (thay vì ~22) nhưng vẫn dừng ở .0955; full chỉ sớm hơn ít; same time = 0.

Vai trò của temporal alignment trên UCI





same time một token cho mọi slot **no time** bỏ token, giữ thứ tự sự kiện **per-ego permute** giữ phân đoạn, xáo nhãn riêng từng ego

Không biến thể nào khác **full** có ý nghĩa thống kê (mọi CI 95% chứa 0) → trên UCI, temporal token và căn chỉnh thời gian giữa các ego **không có tác dụng đo được**.

4. THỰC NGHIỆM · TÁI LẬP TABLE 4

Temporal alignment: chúng tôi so với paper

Biến thể	UCI (chúng tôi)	paper	Δ so với full [CI 95%]	ML-10M (mã gốc)	paper
full	**.104 ± .017**	.104	–	.099 ± .061	.138
same time	**.104 ± .017**	.090	−.001 [−.016, +.015]	.118 ± .062	.147
no time	**.109 ± .011**	.111	+.005 [−.009, +.018]	.118 ± .062	.117
per-ego permute	**.097 ± .022**	–	−.007 [−.029, +.014]	không chạy	–

NDCG@5 chuẩn, mean ± std trên 10 seed. Δ so cặp cùng seed; sign test p ≥ .75 cho cả ba so sánh trên UCI.

PAPER KẾT LUẬN

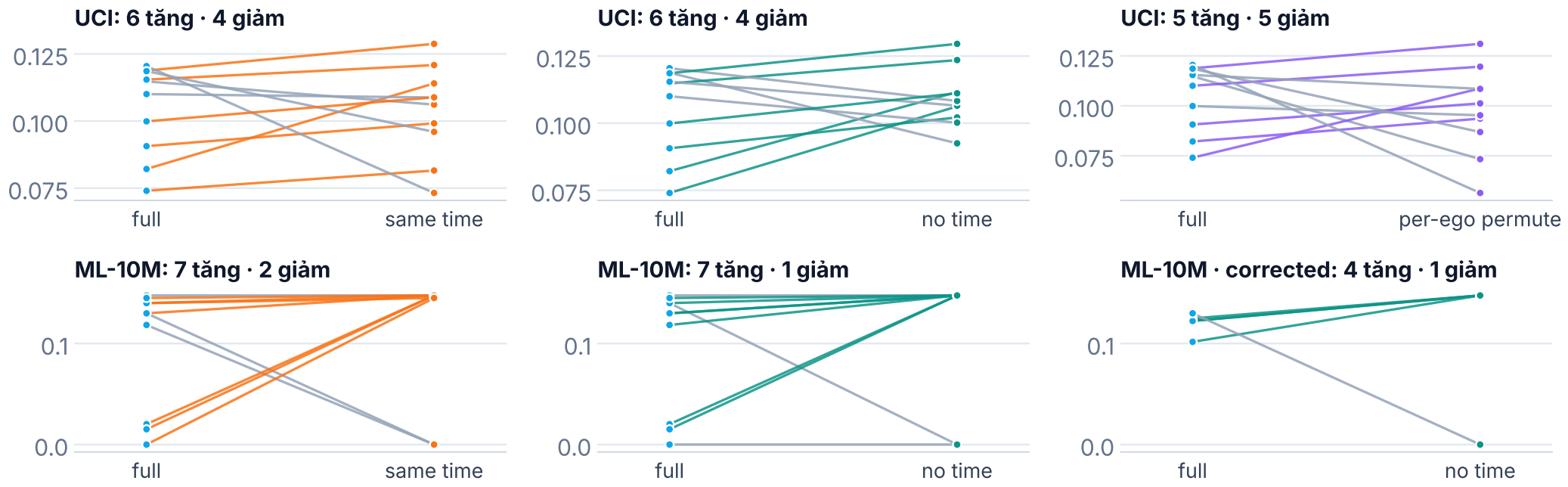
Temporal alignment quan trọng với UCI và ML-10M; same time kém nhất trên UCI (.090).

CHÚNG TÔI THẤY

UCI: thứ tự trung bình giống paper (no time ≥ full ≥ same time) nhưng **nằm trong nhiễu seed**. ML-10M: so sánh không hợp lệ vì mọi biến thể suy biến.

PHÂN TÍCH BỔ SUNG · SO CẶP THEO SEED

Từng seed thay đổi thế nào giữa các biến thể?



UCI (HÀNG TRÊN)

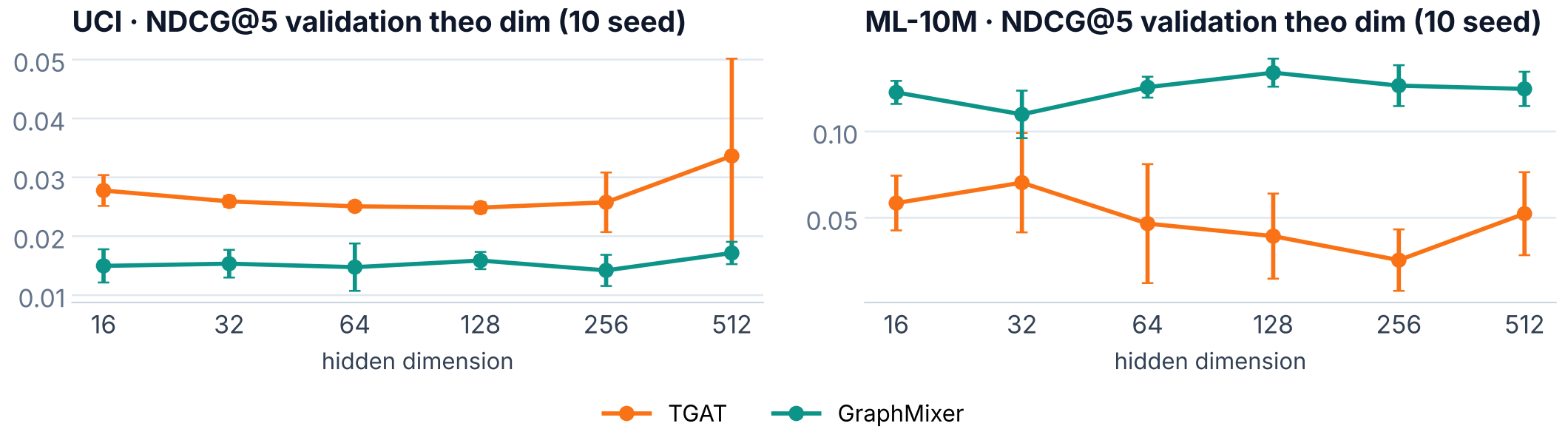
Các đường đan chéo nhau, số seed tăng ≈ số seed giảm → chênh lệch là nhiễu giữa seed, khớp với CI 95% chứa 0 ở slide 23.

ML-10M (HÀNG DƯỚI)

Seed chỉ nhảy giữa hai mức: **0** (không sinh dự đoán) và **.147** (đoán phim 1419) → so biến thể = đếm seed rơi vào mức nào.

PHÂN TÍCH BỔ SUNG · HIDDEN DIMENSION CỦA BASELINE

Chọn dimension: tốn kém nhưng ít tác dụng



Gần như phẳng

Khác biệt giữa các dimension phần lớn nằm trong sai số giữa seed; dim lớn không cho kết quả tốt hơn rõ rệt.

Nhưng rất tốn

GraphMixer ML-10M dim 512 mất **24,5 giờ**/seed (dim 16: 1,8 giờ) — hơn một nửa trong 42 giờ/lượt ở slide 15.

Validation ≠ test

UCI: baseline chỉ đạt .015–.034 trên validation nhưng .05–.07 trên test → chọn theo validation khá nhiễu.

TỔNG KẾT · KẾT QUẢ ĐẠT ĐƯỢC

Tổng kết các kết quả đã chạy

Hạng mục	Kết quả	Đánh giá
Dữ liệu	UCI và ML-10M từ bản preprocess của tác giả; kiểm tra hash, split và khớp với đồ thị gốc. 125/125 lượt chạy hoàn tất, ≈ 689 GPU-giờ.	Hoàn tất
Link prediction · UCI	SimpleDyG **.104 ± .017** = paper (.104), tốt hơn TGAT (.049) và GraphMixer (.070).	Khớp paper
Link prediction · ML-10M	.099 ± .061 so với paper .138. Mô hình suy biến về phim phổ biến nhất (trần .148); mask padding giúp ổn định (std .011) nhưng không khắc phục được.	Lệch · đã tìm nguyên nhân
Baseline kiểm chứng	Heuristic "lặp lại slot gần nhất" đạt **.293** trên UCI, ~2,8× mọi mô hình.	Phát hiện mới
Temporal alignment	4 biến thể × 10 seed trên UCI; Δ so với full: same time −.001, no time +.005, per-ego permute −.007, mọi CI 95% chứa 0.	Không có hiệu ứng đo được

Tóm lại: tái lập được kết quả chính trên UCI; kết quả ML-10M và vai trò của temporal alignment **không được xác nhận** trong phạm vi thực nghiệm này.

NHẬN XÉT · ĐỒ ÁN

Nhận xét, hạn chế và hướng phát triển

1 Đơn giản và rẻ

Transformer gốc tái lập được trên UCI, chỉ tốn < 10% GPU-giờ so với baseline; GraphMixer ML-10M mất 42 giờ/lượt.

2 Đánh giá cần baseline đơn giản

Heuristic không huấn luyện vượt mọi mô hình trên UCI; kết luận "vượt trội" của paper cần đặt cạnh baseline này.

3 Temporal token chưa quyết định

Bỏ hoặc xáo token thời gian không làm kết quả UCI thay đổi đáng kể; tín hiệu chính là tính lặp lại của từng ego.

4 Cần kiểm tra suy biến

Trên ML-10M, điểm số phản ánh độ phổ biến của phim, kể cả số của paper (same time .147 ± .001).

HẠN CHẾ

- 2/4 dataset, 2/8 baseline (adapter BPR, feature 0).
- 10 seed chỉ phát hiện được hiệu ứng ≳ .015 NDCG; ML-10M corrected mới chạy 5 seed.
- Nhận định về số ML-10M của paper là suy luận, chưa kiểm chứng bằng checkpoint của tác giả.

HƯỚNG PHÁT TRIỂN

- Đưa baseline "lặp lại / phổ biến" thành chuẩn khi đánh giá.
- ML-10M: tăng lr hoặc kích thước mô hình, phạt node phổ biến khi decode.
- Thêm MMConv, Hepth; thử alignment thích nghi theo ego.

Thank you

Questions & Discussion

Yuxia Wu, Yuan Fang, Lizi Liao (2024). On the Feasibility of Simple Transformer for Dynamic Graph Modeling. Proceedings of the ACM Web Conference 2024 (WWW '24), Singapore. doi:10.1145/3589334.3645622